Glava 3

OpenMP: petlje i redukcije

Cilj vežbe

- Pokrenuti paralelni region i razlikovati deljene i privatne podatke.
- Proveriti sabiranje pomoću kritične sekcije, atomskog ažuriranja i redukcije.
- Predvideti dodelu iteracija i ispitati ravnotežu opterećenja.
- Paralelizovati direktnu pokretnu srednju vrednost i izmeriti ubrzanje.

OpenMP opisuje paralelni rad direktivama, dok izvršna biblioteka upravlja nitima i podelom iteracija. Za razliku od ručne podele u vežbi 2, programer ne kreira svaku nit zasebno. I dalje mora odrediti koji podaci se dele i koji pristupi zahtevaju zaštitu. Sve komande pokretati iz `root_dir`. Celi primeri su u `src/openmp/`; prikazani isečci izdvajaju samo mehanizam koji se ispituje.

3.1 Paralelni region

```
g++ -O3 -Wall -Wextra -fopenmp \
    src/openmp/hello.cpp -o src/omp_hello
./src/omp_hello 4
```

Primer koristi sledeći region:

```
#pragma omp parallel num_threads(requested_num_workers)
{
    int worker_id = omp_get_thread_num();
    int num_workers = omp_get_num_threads();

    printf("Hello from thread %d of %d threads.\n", worker_id, num_workers);
}
```

Direktiva pokreće tim, a svaka nit izvršava telo regiona. Lokalne promenljive unutar tela pripadaju pojedinačnoj niti. Broj niti je zahtev izvršnoj biblioteci; stvarnu veličinu tima proveriti u ispisu. Indeks niti označava njeno mesto u timu, od 0 do veličine tima umanjene za 1. Na kraju regiona glavna nit nastavlja kada sve niti završe. Redosled ispisa zavisi od toka izvršavanja.

Pokrenuti primer za 1, 2 i 4 niti. Prebrojati poruke niti i uporediti indekse sa prijavljenom veličinom tima. Za ponovljiv izbor tima može se zadati:

```
export OMP_DYNAMIC=FALSE
export OMP_NUM_THREADS=4
export OMP_DISPLAY_ENV=TRUE
./src/omp_hello 4
unset OMP_DISPLAY_ENV
```

U ovom primeru klauzula `num_threads(requested_num_workers)` zadaje traženi broj. Za ostale primere bez te klauzule koristi se podešavanje okruženja. Zabeležiti i ograničenja radnog računara ako stvarni tim odstupa.

3.2 Deljeni i privatni podaci

Za petlju koja sabira niz, ulaz može biti deljen jer ga niti samo čitaju. Indeks paralelne petlje je privatan. Jedna obična suma koju svi menjaju stvara trku. Sledeći isečak je namerno neispravan:

```
long long sum = 0;

#pragma omp parallel for
for (long long i = 1; i <= n; ++i)
{
    sum += i;
}
```

Za dve niti koje dodaju 1 i 2 u početnu nulu napisati mogući redosled čitanja i upisa u kojem jedno ažuriranje nestaje. To je ilustracija sukoba; u C++ trka znači nedefinisano ponašanje. Barijera na kraju petlje čeka niti, ali ne štiti ažuriranja tokom rada. Netačnu verziju ne koristiti kao bazu.

3.3 Tri načina sabiranja

Izvori `sum_critical.cpp`, `sum_atomic.cpp` i `sum_reduction.cpp` u direktorijumu `src/openmp/` sabiraju brojeve od 1 do N. Svaki proverava rezultat formulom $N(N+1)/2$. Bitna razlika je mesto i učestalost sinhronizacije:

```
#pragma omp parallel for
for (long long i = 1; i <= n; ++i)
{
#pragma omp critical
    sum += i;
}

#pragma omp parallel for
for (long long i = 1; i <= n; ++i)
{
#pragma omp atomic
    sum += i;
}

#pragma omp parallel for reduction(+: sum)
for (long long i = 1; i <= n; ++i)
{
    sum += i;
}
```

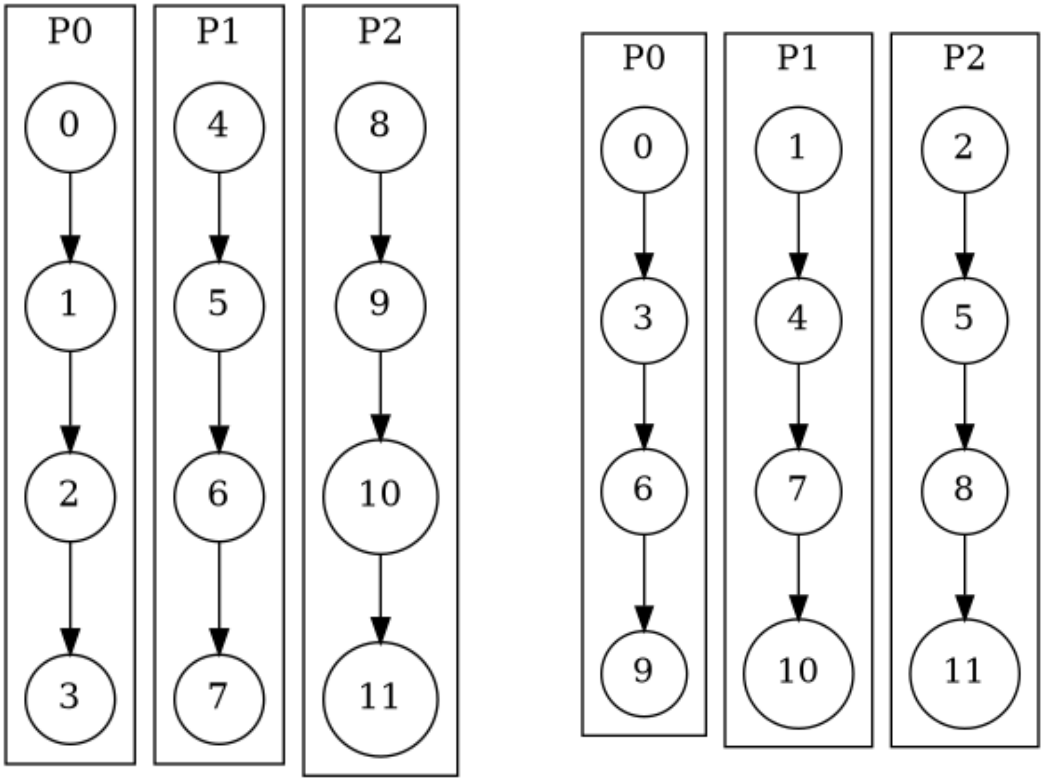
Kritična sekcija dopušta jednu nit u zaštićenom delu. Atomska direktiva štiti jedno ažuriranje. Redukcija daje nitima privatne parcijalne sume, koje se na kraju spajaju sa početnom vrednošću zajedničke sume. Za sabiranje privatne kopije počinju od nule. Pre svakog nezavisnog merenja ponovo postaviti zajedničku sumu na nulu.

```
g++ -O3 -Wall -Wextra -fopenmp \
    src/openmp/sum_critical.cpp -o src/omp_sum_critical
g++ -O3 -Wall -Wextra -fopenmp \
    src/openmp/sum_atomic.cpp -o src/omp_sum_atomic
g++ -O3 -Wall -Wextra -fopenmp \
    src/openmp/sum_reduction.cpp -o src/omp_sum_reduction
OMP_NUM_THREADS=4 ./src/omp_sum_critical 10
OMP_NUM_THREADS=4 ./src/omp_sum_atomic 10
OMP_NUM_THREADS=4 ./src/omp_sum_reduction 10
```

Za ovaj ulaz sva tri rezultata moraju biti 55. Proveriti i $N = 0$, $N = 1$ i N manje od broja niti. Predvideti gde postoji jedna često menjana lokacija, a gde se veći deo rada odvija nad lokalnim stanjem. Ispravnost ne garantuje isto vreme izvršavanja. Za `double` redukciju redosled sabiranja može biti različit, pa rezultat proveravati uz obrazloženu toleranciju.

3.4 Raspoređivanje iteracija

Za 12 iteracija i tri niti ručno dodeliti indekse za `schedule(static,4)` i `schedule(static,1)`. Kod prve podele svaka nit dobija jedan blok od četiri indeksa; kod druge se pojedinačni indeksi dodeljuju ciklično. Bez navedene veličine bloka statička podela obično daje približno jednake uzastopne blokove; tačne granice nisu prenosiva pretpostavka.



blokovska podela

ciklična podela

Slika 3.1: Statička podela iteracija.

Dinamička podela dodeljuje sledeći blok slobodnoj niti. Vođena podela počinje većim blokovima i postepeno ih smanjuje do zadatog minimuma, osim mogućeg poslednjeg manjeg bloka. Njihova stvarna dodela zavisi od toka izvršavanja. Za proveru dodele pokrenuti priloženi program `src/openmp/schedule.cpp`:

```
g++ -O3 -Wall -Wextra -fopenmp \
    src/openmp/schedule.cpp -o src/omp_schedule
OMP_NUM_THREADS=3 OMP_SCHEDULE=static,4 ./src/omp_schedule
OMP_NUM_THREADS=3 OMP_SCHEDULE=static,1 ./src/omp_schedule
```

Program beleži vlasnika svake iteracije, a ispisuje ga posle petlje. Uporediti ispis sa ručnom podelom. Promenljiva `OMP_SCHEDULE` važi za petlje sa `schedule(runtime)`. Za merenje se koristi drugi program, bez beleženja svakog vlasnika i bez ispisa iz niti.

3.5 Zadatak: pokretna srednja vrednost u OpenMP-u

Dopuniti funkciju u fajlu `src/average/openmp.cpp`. Ona dobija ulaz `x`, dužine prozora `widths` i izlaz `y`. Za početak su sve dužine prozora jednake, pa funkcija računa isti zadatak kao u prethodne dve vežbe. Proračun jednog izlaza izgleda ovako:

```
int m = y.size();
for (int i = 0; i < m; ++i)
{
    double sum = 0;
    for (int j = 0; j < widths[i]; ++j)
    {
        sum += x[i + j];
    }
    y[i] = sum / widths[i];
}
```

Podeliti spoljašnju petlju direktivom `omp for schedule(runtime)` unutar jednog paralelnog regiona. Suma ostaje unutar petlje, pa svaka nit ima svoju. Ulazni niz i dužine prozora se samo čitaju, a svaki izlazni element upisuje jedna nit. Svaki prozor daje zaseban rezultat.

Funkcija treba da vrati stvarni broj niti, da bi program ispravno izračunao efikasnost. Okvir regiona može se zapisati ovako:

```
int num_workers = 1;
#pragma omp parallel shared(num_workers)
{
#pragma omp single
    num_workers = omp_get_num_threads();

    // TODO: add omp for schedule(runtime) and the output loop.
}
return num_workers;
```

Direktiva `single` daje jednoj niti da zabeleži veličinu tima. Sve niti zatim dolaze do radne petlje. Broj niti čitamo po završetku regiona. Direktiva `for` deli iteracije postojećem timu; bez nje bi svaka nit izvršavala celu petlju i sukobila se sa drugim upisima. Po želji dodati `default(none)` i izričito navesti deljene promenljive.

```
g++ -O3 -Wall -Wextra -fopenmp \
    src/average/openmp_experiment.cpp src/average/sequential.cpp \
    src/average/openmp.cpp -o src/omp_average
./src/omp_average --check
./src/omp_average regular 8 3 4 static omp_8_3.csv
```

Ime CSV fajla može se dodati posle izbora rasporeda. Kada se izostavi, koristi se `omp_average.csv`. Provere i sažetak ispisuju se na konzoli, a poruke o greškama idu kroz `cerr`. Prva komanda uključuje OpenMP podršku. Provere koriste poznate male rezultate, negativne vrednosti i više niti nego izlaza. Male testove možete dodati u funkciju `small_checks` u `src/average/openmp_experiment.cpp`. Za ulaz $[1, 2, 3, 4, 5, 6, 7, 8]$ i prozor 3 rezultat je

$[2, 3, 4, 5, 6, 7]$. Za velike ulaze svaki element se poredi sa sekvencijalnim direktnim postupkom. Sekvencijalni kod iz prve vežbe ostaje u `src/average/sequential.cpp`. Ne stavljati paralelnu direktivu na ažuriranje jedne klizne sume: sledeće ažuriranje zavisi od prethodnog.

3.6 Dva kratka eksperimenta

3.6.1 Jednak rad: uticaj broja niti

Za $N = 1000000$ i $W = 64$ izmeriti statičku podelu sa 1, 2 i 4 niti, do broja CPU-a dostupnih procesu. Prvo predvideti da li je svaka nit dobio približno isti broj sabiranja. Može li i pored toga ubrzanje biti manje od broja niti?

3.6.2 Nejednak rad: izbor podele

Ista funkcija može računati različite dužine prozora. Priloženi program priprema prozore koji postepeno rastu od 1 do $W_{\max}$ i sekvencijalnu referencu za njih. Na primer, za četiri izlaza i $W_{\max} = 4$ dužine su $[1, 2, 3, 4]$. Za ulaz $[1, 2, 3, 4, 5, 6, 7]$ očekivani izlazi su $[1, 2.5, 4, 5.5]$. Proveriti ih ručno: izlaz i uvek počinje od `x[i]`.

Ovde N označava broj izlaza, a ulaz ima $N + W_{\max} - 1$ elemenata, tako da svi prozori staju. Za jedan izlaz koristi se prozor 1. Koristiti priloženi generator, sekvencijalni postupak i merenje.

Za $N = 100000$ i $W_{\max} = 256$ koristiti isti broj niti u tri poređenja: `static`, `static,1` i `dynamic,64`. Uzeti najveći broj niti iz prethodnog eksperimenta. Predvideti koji statički blok ima najviše rada. Zašto ciklična podela može da pomogne iako svaka nit i dalje dobija približno isti broj izlaza?

```
python3 scripts/run_experiments.py 3 --out results/lab3
python3 scripts/plot_results.py results/lab3
```

Skripta kompajlira program, proverava ga i pokreće oba eksperimenta. Za manju mašinu može se dodati `-max-workers 2`. Za novu seriju izabrati novi direktorijum. Program može da se pokrene i zasebno:

```
./src/omp_average irregular 100000 256 4 all irregular.csv
./src/omp_average regular 1000000 8 4 static regular.csv
```

Opcija `all` ispituje navedene tri strategije, menjajući njihov redosled između ponavljanja. Ovaj program postavlja strategiju iz argumenta komandne linije; `OMP_SCHEDULE` se koristi u prethodnom programu za prikaz vlasnika. Program isključuje automatsko menjanje veličine tima i beleži stvarni broj niti. Ostala OpenMP podešavanja sačuvati uz rezultate.

3.7 Kako čitati rezultate

Svaka verzija se prvo izvrši za zagrevanje u istom procesu, a zatim pet puta za merenje. Program smenjuje redosled verzija i čuva sva vremena. Meri se poziv proračuna: ulazak u region, podela rada, računanje i završno čekanje. Uključeno je i kratko beleženje veličine tima u regionu. Priprema nizova, izbor strategije, provera i ispis su van tog intervala. OpenMP može da ponovo koristi već stvorene niti; ovo su merenja posle zagrevanja, dok C++ verzija iz druge vežbe svaki put stvara nove niti.

Tabela `summary.csv` sadrži srednju vrednost, minimum i maksimum. Za paralelne verzije prikazuje i $S_p = T_{\text{direct}}/T_p$ i $E_p = S_p/p$, gde je p stvarni broj niti. Za nejednake prozore baza računa upravo te prozore.

Kod jednakih prozora pogledati i grafik `ratio_vs_sliding.png`: on prikazuje T_{sliding}/T_p. Vrednost ispod 1 znači da je sekvencijalni klizni postupak brži. To je poređenje algoritama, pa taj odnos ne delimo brojem niti da bismo dobili paralelnu efikasnost. Dobra podela skupljeg algoritma ne mora pobediti jednostavniji postupak koji ukupno obavlja manje rada.

Za samostalnu proveru promeniti prozor sa 64 na 8 ili maksimalni nejednaki prozor sa 256 na 64. Pre pokretanja zapisati predviđanje, pa proveriti ispravnost i ponoviti merenje za isti broj niti. Za dodatno istraživanje ispitati `guided,64`.

3.8 Kako nit uzima sledeći zadatak

Četiri nezavisna zadatka traju 8, 1, 1 i 1 vremensku jedinicu. Ovo su zadate cene u modelu. Dve niti dobijaju po dva uzastopna zadatka. Izračunati kada završava svaka nit. Zatim rasporediti zadatke tako da slobodna nit uzima sledeći dostupan. Koji zadatak sada određuje najkraće moguće vreme?

Istu ideju možete opisati C++ nitima. Za nepromenljiv spisak zadataka, zajednički indeks `next`, početno 0, i jedan mutex, svaka nit ponavlja:

```
lock the mutex
if no tasks remain:
    unlock and finish
take task number next; increase next
unlock the mutex
compute the task using private working data
```

Objasniti zašto uzimanje broja mora biti zaštićeno, a nezavisni proračun treba da bude izvan zaključavanja. Šta bi se promenilo kada bi nit uzimala više zadataka odjednom? Odgovor obrazložiti na modelu.

3.9 Pitanja o arhitekturi procesora

1. Zašto više niti može usporiti kratak proračun?

2. Koja lokacija se stalno menja kod atomskog sabiranja, a gde se čuvaju parcijalni rezultati redukcije?

3. Zašto ravnomernija podela rada ne mora dati kraće vreme? Razmotriti cenu dodele blokova i pristupe susednim elementima.

4. Da li je vaš paralelni direktni postupak brži od kliznog sekvencijalnog? Šta to govori o izboru algoritma?

Rezultat vežbe

- Pokazati ispravnu redukciju i ručnu dodelu 12 iteracija na tri niti.
- Pokazati sopstvenu OpenMP funkciju i proverene jednake i nejednake prozore.
- Objasniti grafik ubrzanja i poređenje tri strategije; proveriti novi parametar.
- Na modelu četiri zadatka objasniti izbor posla i granicu zaključavanja.